

Aufgabenblatt: Protokolle und Schnittstellen

Modul 321 – Lektion 08

Name: _____

Datum: _____

Ziel der Lektion: Sie können wichtige Protokolle und Schnittstellenstile benennen, deren Aufgaben erklären und für einfache Szenarien sinnvoll auswählen.

Hinweise

- Antworten Sie in ganzen Sätzen oder präzisen Stichworten.
- Begründen Sie Zuordnungen kurz und fachlich sauber.
- Skizzen und kleine Übersichten sind erlaubt.

1. Begriffe erklären

Erklären Sie die folgenden Begriffe in **1 bis 3 eigenen Sätzen**.

Begriff	Ihre Erklärung
TCP	
HTTP	
REST	
GraphQL	

gRPC	
WebSocket	
Gateway	
Proxy	

2. Beispiele zuordnen

Ordnen Sie jedem Beispiel den passendsten Begriff zu:

TCP , HTTP , REST , GraphQL , gRPC , WebSocket , Gateway , Proxy

1. Ein Client sendet eine GET-Anfrage an `/api/products/5` und erhält JSON zurück.
2. Zwei Microservices tauschen intern binär kodierte Nachrichten mit automatisch generiertem Code aus.
3. Eine Chat-Anwendung empfängt neue Nachrichten sofort, ohne die Seite neu zu laden.
4. Ein Client sendet eine Anfrage und fragt gezielt nur nach Name und E-Mail, nicht nach allen Feldern.
5. Alle Anfragen laufen über einen einzigen Eintrittspunkt, der Authentifizierung und Routing übernimmt.
6. Verlorene Datenpakete werden automatisch erneut gesendet, bis sie ankommen.
7. Ein Webserver im Firmen-Netzwerk leitet Anfragen der Mitarbeitenden ans Internet weiter.
8. Eine Antwort enthält den Status-Code 404 Not Found.

3. Protokolle und Schnittstellen vergleichen

Frage	REST	GraphQL
Wie viele Endpunkte hat die Schnittstelle typischerweise?		
Wer bestimmt die Struktur der zurückgegebenen Daten?		
In welchem Szenario ist dieser Stil besonders geeignet?		

Frage	Gateway	Reverse Proxy
Welche Hauptaufgabe hat diese Komponente?		
Was macht sie für das System einfacher?		
Warum sollte man beide nicht verwechseln?		

4. Aussagen beurteilen

Beantworten Sie mit **richtig** / **falsch** und begründen Sie kurz.

1. REST und HTTP sind dasselbe.
2. GraphQL verwendet für jede Ressource einen eigenen Endpunkt.
3. gRPC ist immer schneller als REST, unabhängig vom Anwendungsfall.
4. Ein API-Gateway kann Authentifizierung zentral übernehmen.
5. WebSockets eignen sich besonders gut für einfache einmalige Leseanfragen.
6. TCP garantiert, dass Pakete in der richtigen Reihenfolge ankommen.

5. Szenarien analysieren

Beschreiben Sie für jedes Szenario, welche Schnittstelle oder welches Protokoll am besten passt, und begründen Sie kurz.

1. Ein Entwicklungsteam baut eine öffentliche API für einen Online-Shop. Produkte sollen gelesen, erstellt und gelöscht werden können.

2. Eine mobile App zeigt Benutzerprofil, Bestellungen und Empfehlungen an. Je nach Ansicht werden unterschiedliche Daten benötigt.
3. Zwei interne Microservices müssen sich bei jeder Bestellung gegenseitig aufrufen. Performance und Typsicherheit sind wichtig.
4. Eine kollaborative Textedit-Anwendung soll Änderungen sofort an alle Mitarbeitenden übermitteln.
5. Ein Unternehmen betreibt zehn verschiedene Backend-Dienste. Alle Clients sollen nur eine einzige Adresse kennen.

6. Schnittstellen einordnen

Füllen Sie die Tabelle aus.

Stil	Datenformat	Kommunikationsmuster	Typischer Einsatz
REST			
GraphQL			
gRPC			
WebSocket			

7. Mini-Entwurf

Entwerfen Sie für eine einfache Lieferdienst-Anwendung eine grobe Schnittstellenübersicht.

Funktionen:

- Benutzer melden sich an und verwalten ihre Bestellungen.
- Eine mobile App soll genau die Daten laden, die sie gerade braucht.
- Interne Dienste (Lager, Bezahlung, Versand) müssen schnell miteinander kommunizieren.
- Fahrer erhalten Live-Updates zu neuen Aufträgen.
- Alle externen Anfragen sollen über einen einzigen Zugangspunkt laufen.

1. Welche Schnittstellenstile oder Protokolle würden Sie einsetzen?
2. Begründen Sie Ihre Wahl für jeden Bereich kurz.
3. Welche Komponente steuert den Zugang von aussen?

8. Reflexion

Beantworten Sie die Fragen kurz.

1. Warum ist es wichtig, dass Schnittstellen klar definiert und dokumentiert sind?
2. Warum sollte man nicht immer die neueste oder komplexeste Schnittstellentechnologie wählen?
3. Was ist der Unterschied zwischen einem Protokoll und einem Architekturstil wie REST?